

UNIDAD 2 — LA VPS: APROVISIONAR, ENTRAR Y CERRAR

Clase 2 · La VPS: aprovisionar, entrar y cerrar

Ejercicios de teoría — CLAVE DE CORRECCIÓN

Programación 3 — Tecnicatura Universitaria en Programación — UTN FRM

Modalidad	En grupos de dos o tres. Se responde con el apunte de la clase a mano.
Tiempo sugerido	50 a 60 minutos.
Puntaje total	100 puntos. Partes A (20), B (10), C (30), D (30) y E (10).
Qué se evalúa	Que puedan nombrar el concepto que explica cada síntoma, no que recuerden comandos de memoria.
Uso de esta clave	Documento del docente. Cada respuesta cierra con la sección del apunte donde se justifica.

Cómo usar esta clave. Las respuestas están redactadas completas para que puedan entregarse tal cual como devolución. En las partes C, D y E no se busca la redacción textual sino que aparezcan los conceptos marcados en negrita; si el grupo nombra el concepto correcto con sus palabras, el ítem está bien.

Parte A — Verdadero o falso, con justificación

20 puntos (2 por ítem)

Indicá si cada afirmación es verdadera o falsa. **La justificación es obligatoria:** una respuesta sin justificar no suma puntaje. Una línea alcanza.

- A1.** Si el VPS queda comprometido, el proveedor puede auditar lo que pasó adentro del sistema operativo y ayudar a resolverlo.

FALSO. El proveedor responde por el hipervisor, el hardware y la red. Todo lo que ocurre dentro del sistema operativo huésped —usuarios, puertos, actualizaciones, servicios expuestos— es del titular. Y ese corte no es una postura comercial: es la consecuencia directa de que el proveedor, *por diseño*, no puede ver adentro. El aislamiento que te protege del vecino es el mismo que le impide mirar al proveedor.

Apunte: §2.2 y §2.3

- A2.** En la autenticación por clave pública, la clave privada viaja cifrada hasta el servidor para que este pueda compararla con la pública.

FALSO. La clave privada **nunca se transmite, ni siquiera cifrada**. La autenticación es un desafío criptográfico: el servidor manda un dato aleatorio, el cliente lo *firma* con su clave privada, y el servidor *verifica* esa firma con la clave pública que tiene guardada. De ahí la propiedad más importante del esquema: si el servidor entero cae en manos ajenas, el atacante se lleva un `authorized_keys` lleno de claves públicas que no le sirven para nada.

Apunte: §2.5.1

- A3.** La directiva `PermitRootLogin prohibit-password` impide entrar al servidor como root.

FALSO. Prohíbe entrar como root **con contraseña**. El acceso por clave sigue perfectamente permitido —de hecho es el valor predeterminado en las distribuciones modernas y es el adecuado para este práctico, donde el trabajo administrativo es constante. El nombre de la directiva dice exactamente lo que hace; hay que leerlo entero.

Apunte: §2.5.5

- A4.** Un servicio que escucha en `127.0.0.1` es inalcanzable desde internet aunque el firewall esté apagado y el puerto abierto.

VERDADERO. No hay firewall que valga porque no hay nada que filtrar: sencillamente no hay nadie escuchando del lado de afuera. Es la forma más barata y más robusta de cerrar algo, y es economía del mecanismo en estado puro: no hay ninguna regla que auditar ni que se pueda configurar mal.

Apunte: §2.6.1 y §2.12.1

- A5.** Si `sudo ufw status` informa que el puerto 5432 está bloqueado, entonces el puerto 5432 está efectivamente cerrado.

FALSO. Un contenedor levantado con `-p 5432:5432` queda accesible desde internet igual, y `ufw status` sigue informando que está bloqueado. Las dos cosas son ciertas: `ufw` *escribió* esa regla, y esa regla *no se evalúa nunca* para ese tráfico. La herramienta te informó sobre sí misma y vos lo leíste como información sobre el sistema.

Apunte: §2.8 y §2.8.1

- A6.** Un servidor web necesita un puerto distinto por cada conexión simultánea que atiende.

FALSO. Una conexión establecida no se identifica por un puerto sino por una **tupla de cuatro elementos**: dirección de origen, puerto de origen, dirección de destino y puerto de destino. Dos conexiones al mismo servidor y al mismo puerto no se confunden porque difieren en el par de origen. Por eso un único puerto 443 atiende diez mil conexiones: no hay diez mil puertos, hay diez mil tuplas.

Apunte: §2.6.1

- A7.** Si toda la aplicación funciona por HTTPS, el puerto 80 se puede cerrar sin consecuencias.

FALSO. Let's Encrypt valida la titularidad del dominio pidiendo un archivo **por el puerto 80** (desafío HTTP-01). Sin ese puerto abierto no hay certificado, y sin certificado no hay HTTPS. El 80 no está abierto para servir la aplicación: está abierto para redirigir a HTTPS y para que se emita y se renueve el certificado.

Apunte: §2.7.2

- A8.** Que Docker se saltee ufw es un error conocido de Ubuntu que se corrige actualizando el sistema.

FALSO. No es un error de Docker ni una mala configuración de Ubuntu: es la **interacción previsible** de dos programas que escriben reglas en el mismo subsistema (netfilter) sin conocerse entre sí, en puntos de enganche distintos. ufw escribe en INPUT; el tráfico hacia los contenedores pasa por FORWARD. Docker incluso documenta la cadena DOCKER-USER como el lugar previsto para intervenir; lo que no hace es avisar que ufw no la usa.

Apunte: §2.8.1

- A9.** En un sistema Unix, poner un proceso a escuchar en el puerto 3000 requiere privilegios de administrador.

FALSO. La restricción de privilegios aplica a los puertos **por debajo de 1024** (los bien conocidos). El 3000 está en el rango de puertos registrados (1024–49151) y lo puede abrir cualquier usuario. Esa restricción histórica —pensada para que un usuario común no pudiera hacerse pasar por el servidor web de la máquina— es justamente la razón por la que las aplicaciones escuchan en 3000, 5000 u 8000 y hay un proxy inverso adelante ocupando el 80 y el 443.

Apunte: §2.6.2

- A10.** Con fail2ban instalado, el servidor queda protegido contra un atacante que quiera entrar por SSH.

FALSO. fail2ban **no impide un ataque dirigido**: quien tenga paciencia puede espaciar los intentos por debajo del umbral, o rotar direcciones de origen. Lo que hace es elevar el costo del ataque automatizado indiscriminado —que es la inmensa mayoría del tráfico hostil— y mantener legibles los registros. Es *una capa más*, no una defensa: defensa en profundidad significa que ninguna capa se asume infalible.

Apunte: §2.11.1 y §2.12.1

Parte B — Opción múltiple

10 puntos (1 por ítem)

Marcá **una sola** opción por ítem.

- B1.** ¿Qué hace que el aislamiento entre dos VPS alojados en el mismo servidor físico sea real y no una promesa comercial?
- a) Un contrato de nivel de servicio con el proveedor
 - b) Que el procesador hace cumplir por hardware la separación de los espacios de memoria
 - c) Que cada VPS tiene su propia dirección IP pública
 - d) Que el hipervisor cifra el disco de cada cliente

b). El sistema operativo de un cliente no puede ver ni tocar el de otro porque son espacios de memoria separados que el procesador hace cumplir. Lo que **sí** se comparte es el hierro: procesador físico, disco y placa de red. Las dos cosas son ciertas al mismo tiempo, y de ahí sale todo lo demás.

Apunte: §2.2

B2. El fenómeno del «vecino ruidoso» —el servidor se pone lento sin que nada haya cambiado de tu lado— se explica por...

- a) ...una falla del hipervisor
- b) ...el sobreaprovisionamiento: el proveedor vende más núcleos virtuales que núcleos reales
- c) ...la caché negativa del DNS
- d) ...que el disco virtual es una imagen y no un disco real

b). Los proveedores apuestan a que no todos los clientes estarán al máximo a la vez. Cuando esa apuesta falla, el que paga es el que está usando el servidor en ese momento.

Apunte: §2.2

B3. En una conexión SSH, ¿qué se autentica **primero**?

- a) El cliente ante el servidor, con su clave privada
- b) El servidor ante el cliente, presentando su clave de anfitrión
- c) Ambos simultáneamente, en la capa de conexión
- d) Ninguno: SSH solo cifra, la autenticación la hace el sistema operativo

b). La capa de transporte (RFC 4253) establece el canal cifrado y autentica al **servidor** ante el cliente, no al revés. Recién sobre ese canal ya seguro ocurre la capa de autenticación (RFC 4252), donde el cliente demuestra quién es. El orden importa: primero sabés con quién estás hablando, después le mostrás tus credenciales.

Apunte: §2.5.1

B4. El modelo de confianza en el primer uso (TOFU) que usa SSH significa que...

- a) ...la conexión es insegura hasta que se instala un certificado
- b) ...es vulnerable exactamente una vez —la primera— y a partir de ahí detecta cualquier suplantación
- c) ...la clave del anfitrión se verifica contra una autoridad certificadora
- d) ...hay que aceptar la huella cada vez que uno se conecta

b). La primera vez el cliente no tiene forma de saber si esa clave de anfitrión es la legítima: pregunta, y si aceptás la guarda en ~/.ssh/known_hosts. Es un compromiso deliberado. Por eso, cuando después aparece la advertencia grande de que la clave cambió, no es un trámite: o el servidor se reinstaló, o alguien se está interponiendo.

Apunte: §2.5.1

B5. Un paquete que llega desde internet al puerto publicado de un contenedor **no pasa por INPUT** porque...

- a) ...Docker desactiva la cadena INPUT al instalarse
- b) ...tras el DNAT en PREROUTING el destino ya es la IP del contenedor, y la decisión de enrutamiento lo manda por FORWARD
- c) ...ufw lo excluye explícitamente de sus reglas
- d) ...los contenedores usan UDP en lugar de TCP

b). El recorrido es: PREROUTING (Docker reescribe el destino en la tabla nat) » decisión de enrutamiento (el núcleo mira el destino *ya reescrito* y concluye que no es para él) » FORWARD. INPUT y FORWARD son caminos **excluyentes**: un paquete pasa por uno o por el otro, nunca por los dos. Y las reglas de ufw viven en INPUT.

Apunte: §2.7.1 y §2.8.1

B6. Los puertos del rango 49152–65535 se llaman dinámicos o efímeros porque...

- a) ...expiran después de un tiempo de inactividad
- b) ...no los asigna nadie: los toma el sistema operativo, típicamente como puerto de origen
- c) ...solo pueden usarse con privilegios de administrador
- d) ...la IANA los reserva para uso experimental

b). Los otros dos rangos sí tienen trámite ante la IANA: bien conocidos (0–1023, trámite formal) y registrados (1024–49151, trámite simplificado). El puerto de origen que usa tu navegador cuando abris una página sale de ese rango efímero.

Apunte: §2.6.2

B7. En `ss -tlnp`, la bandera `-l` pide...

- a) ...que se muestren los nombres de los servicios en lugar de los números
- b) ...únicamente los sockets en estado LISTEN, es decir los que esperan conexiones nuevas
- c) ...los sockets locales del sistema de archivos
- d) ...un listado largo con más columnas

b). Sin la `-l`, `ss` muestra además todas las conexiones ESTABLISHED en curso, que son muchas más y responden otra pregunta. El socket en escucha es uno; las conversaciones en curso son muchas.

Apunte: §2.6.3

B8. ¿Para qué sirve la bandera `-Pn` de `nmap`?

- a) Para escanear también puertos UDP
- b) Para que no intente primero un ping de descubrimiento y escanee directamente
- c) Para omitir los puertos privilegiados
- d) Para que el escaneo no quede registrado en los logs del destino

b). Muchos proveedores descartan los pings entrantes. Sin esa bandera, `nmap` concluiría que el servidor no existe y ni siquiera escanearía — y el grupo se quedaría creyendo que no hay nada abierto.

Apunte: §2.8.2

B9. Para un PostgreSQL que solo tiene que ser alcanzable por la API del mismo proyecto, la estrategia **preferida** del capítulo es...

- a) No declarar ningún mapeo de puertos: no publicarlo en absoluto
- b) Publicarlo con `-p 5432 : 5432` y bloquearlo con `ufw`
- c) Publicarlo y escribir reglas en la cadena `DOCKER-USER`
- d) Publicarlo con una contraseña larga

a). Un servicio que no publica puertos sigue siendo perfectamente accesible para los demás servicios del proyecto a través de la red interna de Docker (Clase 5). Es economía del mecanismo: no hay regla que auditar, no hay herramienta en la que confiar, no hay nada que se pueda configurar mal. **El puerto más seguro es el que no existe.**

Apunte: §2.8.3 y §2.12.1

B10. La razón principal por la que el capítulo elige Ed25519 en lugar de RSA de 4096 bits es que...

- a) RSA ya fue quebrado criptográficamente
- b) Ed25519 tiene un solo tamaño de clave y por lo tanto muchas menos formas de configurarlo mal
- c) Ed25519 es el único que soporta OpenSSH
- d) RSA no funciona con autenticación por desafío

b). Ofrece además claves más cortas con seguridad equivalente o superior y firmas más rápidas, pero lo decisivo en la práctica es lo otro: RSA admite tamaños inseguros que siguen siendo aceptados por compatibilidad; Ed25519 no tiene perilla que tocar. Es economía del mecanismo aplicada a la criptografía.

Apunte: §2.12.3

Parte C — Desarrollo conceptual

30 puntos (3 por ítem)

Respondé con precisión y brevedad. Se evalúa que uses el vocabulario técnico del capítulo, no la extensión de la respuesta.

C1. Enunciá las tres propiedades que Popek y Goldberg exigieron en 1974 para llamar «virtualizador» a un sistema, y explicá por qué la virtualización tardó treinta años en llegar a los servidores comunes.

Equivalencia (los programas se comportan igual que sobre el hardware real), **control de recursos** (el hipervisor tiene la última palabra sobre el hardware) y **eficiencia** (la mayoría de las instrucciones se ejecutan directamente sobre el procesador, sin emulación).

Popek y Goldberg demostraron formalmente qué condición debe cumplir un juego de instrucciones para admitirlas, y **x86 no la cumplía**. Por eso hubo que llegar por etapas: traducción binaria en tiempo de ejecución (VMware, 1999), modificar el sistema operativo huésped para que colaborase (Xen, 2003) y finalmente soporte del propio procesador (VT-x y AMD-V, ~2006), que es lo que se usa hoy y lo que hace que un VPS rinda casi como una máquina real.

Apunte: §2.2

- C2.** Diferenciá hipervisor de **tipo 1** y de **tipo 2**, dando un ejemplo de cada uno, e indicá cuál usa el VPS del práctico.

Tipo 1 (nativo): corre directamente sobre el hardware. Ejemplos: KVM, Xen, VMware ESXi, Hyper-V. Es lo que usan los proveedores de nube y de VPS.

Tipo 2 (alojado): corre como un programa más dentro de un sistema operativo. Ejemplos: VirtualBox, VMware Workstation. Uso de escritorio y laboratorio.

El VPS del práctico corre sobre uno de **tipo 1**, casi con seguridad **KVM**, que desde 2007 forma parte del propio núcleo de Linux: la máquina física ejecuta Linux, y Linux mismo actúa como hipervisor.

Apunte: §2.2

- C3.** De la arquitectura del VPS se desprenden tres consecuencias prácticas. Enunciá las tres y dá un efecto observable de cada una.

1) El aislamiento es real, pero los recursos se comparten. El procesador hace cumplir la separación de memoria, pero el procesador físico, el disco y la placa son los mismos. Efecto observable: el vecino ruidoso — el servidor se pone lento sin que nada haya cambiado de tu lado.

2) El estado completo de la máquina es un archivo. El disco virtual es una imagen y la configuración es un registro en una base del proveedor. Efecto: cambiar el sistema operativo tarda minutos, y es destructivo y sin deshacer — no se actualiza nada, se tira la imagen anterior y se escribe otra encima.

3) La responsabilidad se corta en un punto preciso. El proveedor responde por hipervisor, hardware y red; todo lo de adentro del sistema operativo es del titular. Efecto: si el servidor queda comprometido, es tu problema — el proveedor no tiene ni jurisdicción ni visibilidad.

Apunte: §2.2

- C4.** El módulo podría haber usado un PaaS y el despliegue tendría menos pasos. Explicá por qué se eligió un VPS, en términos del eje que ordena la tabla de modalidades.

La tabla describe un único eje: **cuánta abstracción hay entre quien desarrolla y la máquina**. Cada escalón hacia arriba elimina trabajo operativo y, con él, elimina también *visibilidad y control*.

Un PaaS resolvería el despliegue en menos pasos pero ocultaría precisamente lo que se pretende enseñar: la red, los puertos, el proxy inverso, los certificados y el aislamiento entre servicios. En un PaaS esas piezas existen —alguien las configuró— pero no son observables, y **lo que no se observa no se aprende**. El VPS es el nivel de abstracción más bajo en el que todavía se puede trabajar en una cursada, y el más alto en el que todas las piezas siguen siendo visibles.

Apunte: §2.3

- C5.** Nombrá las tres capas del protocolo SSH, en orden, y decí qué ocurre en cada una.

1) Capa de transporte (RFC 4253): establece un canal cifrado e íntegro y **autentica al servidor ante el cliente** — el servidor presenta su clave de anfitrión y demuestra que la posee.
2) Capa de autenticación (RFC 4252): ahora sí el **cliente** demuestra quién es. Es acá donde se elige entre contraseña y clave pública.
3) Capa de conexión (RFC 4254): multiplexa sobre el mismo canal la sesión interactiva, la copia de archivos y los túneles de puertos.

El orden es lo que hay que retener: todo lo demás ocurre *sobre* un canal que ya es seguro.

Apunte: §2.5.1

- C6.** Explicá el mecanismo de la autenticación por clave pública y justificá por qué un servidor comprometido no compromete el acceso.

Es un **desafío criptográfico**: el servidor envía al cliente un dato aleatorio, el cliente lo firma con su clave privada, y el servidor verifica esa firma con la clave pública que tiene guardada en `authorized_keys`. **La clave privada nunca se transmite**, ni siquiera cifrada.

Por eso, si el servidor entero cae en manos ajenas, el atacante obtiene claves *públicas*, que solo permiten **verificar** firmas, no producirlas. No le sirven para nada. Con contraseña, en cambio, lo que hay guardado del lado del servidor es un secreto reutilizable, y quien lo obtiene obtiene acceso.

Corolario operativo: por eso la clave privada **nunca** se copia al servidor. Un servidor comprometido con claves privadas adentro no es un servidor comprometido: es todo lo que esas claves abren, comprometido.

Apunte: §2.5.1 y §2.9.3

- C7.** «Un socket se identifica por una tupla, no por un puerto.» Explicá la afirmación y usala para justificar cómo un servidor con un único puerto 443 atiende diez mil conexiones simultáneas.

En la pila TCP/IP una conexión establecida se identifica por **cuatro elementos**: dirección de origen, puerto de origen, dirección de destino y puerto de destino. Dos conexiones al mismo servidor y al mismo puerto no se confunden porque difieren en el par de origen. Diez mil conexiones al 443 son diez mil tuplas distintas, no diez mil puertos.

Además, el **socket en escucha** es un objeto de otro tipo: está a medio especificar, con el origen todavía vacío, y lo que hace es pedirle al sistema operativo «entregame a mí toda conexión nueva dirigida a este número». Cada conexión aceptada genera un socket nuevo, ya completo. Y dos procesos no pueden escuchar a la vez en la misma combinación de puerto e interfaz.

Apunte: §2.6.1

- C8.** Enumerá los cinco puntos de enganche de netfilter y explicá qué papel cumple la **decisión de enrutamiento**.

PREROUTING (apenas llega el paquete, antes de decidir a dónde va), **INPUT** (el paquete va a un proceso de esta misma máquina), **FORWARD** (el paquete atraviesa la máquina hacia otro destino), **OUTPUT** (lo generó un proceso local) y **POSTROUTING** (justo antes de salir por la placa).

La pieza decisiva está entre PREROUTING y los dos siguientes: después de PREROUTING el núcleo mira la dirección de destino y se pregunta **si el paquete es para él o para otro**. Si es para él, sigue por INPUT; si es para otro, sigue por FORWARD. Son **caminos excluyentes**: un paquete pasa por uno o por el otro, nunca por los dos. Toda la sección 2.8 es consecuencia de esto.

Apunte: §2.7.1

- C9.** Enunciá los tres principios de Saltzer y Schroeder que el capítulo destaca y dá **una aplicación concreta de cada uno** tomada de este mismo capítulo.

Mínimo privilegio — todo componente opera con los permisos mínimos necesarios. Aplicación: el contenedor de la base no publica puertos porque no necesita ser alcanzable desde afuera; y la clave pública instalada en el servidor solo permite verificar, no firmar.

Valores predeterminados seguros — lo que no está explícitamente permitido, se niega. Aplicación: `ufw default deny incoming` es literalmente la primera línea del firewall; y es lo que Redis y MongoDB hacían al revés, con los resultados conocidos.

Economía del mecanismo — cuanto más simple es la protección, más fácil es verificar que funciona. Aplicación: «no publicar el puerto» es preferible a «publicarlo y filtrarlo», porque no hay regla que auditar ni herramienta en la que confiar.

Los tres son de **1975**. Las herramientas cambian todo el tiempo; los principios prácticamente nunca.

Apunte: §2.12.1

- C10.** ¿Qué es la **defensa en profundidad**? Nombrá las cuatro capas que tiene este servidor y explicá qué supone cada una respecto de las demás.

Es el concepto operativo de que **ninguna capa se asume infalible**, y por eso se apilan varias independientes: cada una está puesta suponiendo que las otras pueden fallar.

Las cuatro de este servidor: **(1)** autenticación por clave en lugar de contraseña; **(2)** firewall restrictivo (`default deny incoming`); **(3)** no publicar los puertos internos; **(4)** fail2ban.

Ejemplo del razonamiento: la capa 3 existe porque sabemos que la capa 2 **no cubre** el tráfico de Docker (§2.8); y fail2ban existe aunque la capa 1 ya haga improbable la fuerza bruta.

Apunte: §2.12.1

Parte D — Casos de diagnóstico

30 puntos (5 por caso)

Para cada caso indicá: **(a)** el diagnóstico, **(b)** el comando o la evidencia con la que lo confirmarías y **(c)** la corrección. Nombrá el concepto del capítulo que explica el síntoma.

- D1.** Un grupo levantó PostgreSQL en un contenedor con `-p 5432:5432`, corrió `sudo ufw deny 5432` y verificó con `ufw status` que figura bloqueado. Un compañero escanea el servidor desde su casa y el 5432 le aparece **abierto**. **Seguí el recorrido del paquete** y explicá por qué las dos observaciones son correctas.

(a) Diagnóstico: el puerto está publicado por Docker, y ufw no ve ese tráfico.

Recorrido: **1.** El paquete entra por **PREROUTING**, donde Docker tiene una regla en la tabla `nat` que reescribe el destino: donde decía «IP pública del VPS, puerto 5432» ahora dice «IP privada del contenedor, puerto 5432». **2.** Llega la **decisión de enrutamiento**: el núcleo mira el destino ya *reescrito* y concluye que no es para él. **3.** El paquete sigue por **FORWARD**, donde Docker registró sus cadenas `DOCKER` y `DOCKER-USER`. **4. Nunca pasa por INPUT** — y las reglas de ufw viven en `INPUT`.

(b) Evidencia: `nmap -Pn tudominio.me` desde otra máquina. Adentro del servidor, `sudo nft list ruleset` muestra las cadenas de ufw y las de Docker con su punto de enganche.

(c) Corrección: dejar de publicar el puerto (no declarar mapeo) y hablarle por la red interna; o publicarlo solo en local con `-p 127.0.0.1:5432:5432` y llegar por túnel SSH.

Concepto: `INPUT` y `FORWARD` son caminos excluyentes. ufw no miente: informa sobre sus propias reglas, no sobre el estado del sistema.

Apunte: §2.7.1, §2.8.1 y §2.8.3

- D2.** Un grupo conectado por SSH corre `sudo ufw enable` y la terminal se congela. No pueden volver a entrar por más que reintenten.

(a) Diagnóstico: habilitaron el firewall **antes** de permitir el puerto 22. La política predeterminada es `deny incoming`, así que el firewall cortó su propia sesión SSH en el acto y bloquea todo intento nuevo.

(b) Evidencia: la conexión no da «contraseña incorrecta» ni «clave rechazada»: se queda colgada y termina en timeout. Nadie contesta, que es exactamente lo que hace un `deny`.

(c) Corrección: entrar por la **consola de emergencia de Hostinger** —que no pasa por SSH ni por el firewall— y correr `sudo ufw allow 22/tcp`.

Cómo se evita: **permitir primero, habilitar después. Siempre.** El orden de los comandos del capítulo no es estético.

Apunte: §2.7.2 y §2.14

- D3.** De los cuatro integrantes, tres entran al servidor sin problema y el cuarto recibe `Permission denied (publickey)`. Enumerá las causas posibles según el capítulo y cómo distinguirlas.

(a) Diagnósticos posibles: **(i)** falta su clave pública en el servidor — no quedó cargada en `authorized_keys` ni en la sección SSH Keys de hPanel; **(ii)** está usando la clave privada de otro equipo, o generó el par en una máquina y se conecta desde otra.

(b) Cómo distinguirlas: mirar el archivo `~/.ssh/authorized_keys` del servidor desde la sesión de un compañero que sí entra, y comparar contra el contenido del `.pub` del que no entra. Si la línea no está, es (i); si está pero no coincide con la clave del equipo desde el que se conecta, es (ii). `ssh -v` muestra además qué clave está ofreciendo el cliente.

(c) Corrección: agregar la clave pública correcta como una línea más del archivo. **Nunca** compartir la clave privada entre integrantes: el sentido del esquema es que haya cuatro accesos independientes y revocables por separado — con contraseña compartida, a efectos de auditoría, hay una identidad sin dueño.

Apunte: §2.5.3 y §2.14

- D4.** Al conectarse el lunes, un integrante recibe una advertencia grande de que **la clave del anfitrión cambió** y que podría estar ocurriendo un ataque. Un compañero le dice que borre la línea de `known_hosts` y siga. ¿Está bien ese consejo?

(a) Diagnóstico: hay exactamente dos explicaciones: **(i)** el servidor se reinstaló —por ejemplo, se volvió a aplicar la plantilla de Easypanel, que reescribe la imagen y genera claves de anfitrión nuevas—; o **(ii)** alguien se está interponiendo en el camino.

(b) Evidencia: preguntar al grupo si alguien reinstaló o recreó el VPS, y comparar la huella que muestra el cliente con la que informa la consola del proveedor.

(c) El consejo está mal dado tal como está: borrar la línea de `known_hosts` es la *acción* correcta, pero solo **después de verificar el motivo**. Hacerlo por reflejo convierte la única defensa del modelo en un trámite.

Concepto: TOFU — el modelo es vulnerable exactamente una vez, la primera. Todo su valor está en que la advertencia se lea; si se acepta a ciegas, se vuelve vulnerable siempre.

Apunte: §2.5.1 y §2.14

- D5.** Un estudiante dice: «esto es una calculadora de la facultad, no tiene datos de nadie. ¿Quién va a querer hackearla?» Desarmá el razonamiento con los datos del capítulo.

(a) El error de planteo: a nadie le interesa la calculadora. Les interesa **la CPU, el ancho de banda y una dirección IP limpia**. La aplicación es irrelevante; el servidor es el botín.

(b) Evidencia: el descubrimiento es automático, permanente y no selectivo. El espacio IPv4 son ~4.300 millones de direcciones; un solo equipo doméstico emite del orden de un millón de paquetes por segundo, así que barrerlo entero es cuestión de una hora larga —y repartido entre varias máquinas, de minutos. Además el resultado se indexa y se vende (Shodan, Censys): el atacante ni siquiera necesita escanear, le alcanza con consultar. Un VPS recién creado empieza a recibir intentos **dentro de la primera hora**. Comprobación directa: `sudo grep -c "Failed password" /var/log/auth.log`.

(c) Qué se arriesga aunque no haya datos: minado de criptomonedas (consume el 100 % de los recursos y genera la factura), envío de spam (la IP queda en listas negras), participación en ataques de denegación de servicio contra terceros —el servidor pasa a ser el atacante y la responsabilidad es del titular— y **movimiento lateral** si hay claves SSH hacia otros equipos.

Apunte: §2.9.1, §2.9.2 y §2.9.3

- D6.** El grupo configuró `easypanel.tudominio.me` en Settings » Domain, esperó, y el certificado no se emite: el navegador no abre el panel por HTTPS. Indicá las dos causas que contempla el capítulo y cómo distinguirlas.

(a) Diagnósticos posibles: (i) el registro comodín no resuelve — el nombre `easypanel` no fue cargado en ninguna parte y depende del * de la Clase 1; o (ii) el **puerto 80 está cerrado**, y sin él Let's Encrypt no puede validar la titularidad por el desafío HTTP-01.

(b) Cómo distinguirlas: `nslookup easypanel.tudominio.me` — si no devuelve la IP del VPS, es (i) y el problema está en el DNS. Si resuelve bien, `sudo ufw status verbose` y sobre todo `nmmap -Pn tudominio.me` desde afuera para confirmar que el 80 responde.

(c) Corrección: según el caso, corregir el comodín en el panel DNS o abrir el 80. **Y el orden es sagrado:** primero verificar que el dominio nuevo entra, recién después cerrar el acceso directo por el puerto 3000. Al revés, si algo sale mal con el certificado, quedaste sin panel y sin forma de arreglarlo desde el panel.

Apunte: §2.10 y §2.14

Parte E — Lectura de salidas reales

10 puntos

Leer la salida de una herramienta es leer el sistema. Respondé mirando el bloque de cada ítem.

E1.

```
# ss -tlnp
State  Recv-Q  Send-Q  Local Address:Port  Process
LISTEN  0        4096    0.0.0.0:22        users:("sshd",pid=812))
LISTEN  0        4096    0.0.0.0:80        users:("traefik",pid=1544))
LISTEN  0        4096    0.0.0.0:443       users:("traefik",pid=1544))
LISTEN  0        4096    0.0.0.0:3000      users:("docker-proxy",pid=1602))
LISTEN  0        244     127.0.0.1:5432    users:("postgres",pid=1489))
```

(4 puntos) Indicá: (i) cuáles de estos servicios son alcanzables desde internet y cuáles no, y por qué; (ii) cuáles necesita ver el mundo entero; (iii) a qué rango de la RFC 6335 pertenecen el 22, el 3000 y el 5432; (iv) qué te dice el proceso responsable del puerto 3000 sobre si ufw lo protege.

(i) Alcanzables desde la red: 22, 80, 443 y 3000, todos en 0.0.0.0 (todas las interfaces). **No** alcanzable: el 5432, que escucha en 127.0.0.1 — solo procesos del mismo equipo. Esto último vale *aunque el firewall esté apagado*: no hay nadie escuchando del lado de afuera.

(ii) Solo **80 y 443**. El 22 lo necesita el grupo, no el mundo (aunque en este práctico se deja abierto); el 3000 no lo necesita nadie una vez publicado el panel bajo el dominio propio.

(iii) 22 » **bien conocido** (0–1023, trámite formal ante la IANA, y hace falta ser administrador para escuchar ahí). 3000 y 5432 » **registrados** (1024–49151).

(iv) El responsable del 3000 es docker-proxy: el puerto lo publicó **Docker**. Por lo tanto ese tráfico va por FORWARD y **ufw no lo protege**, diga lo que diga ufw status.

Apunte: §2.6.1, §2.6.2 y §2.8.1

E2.

```
# sudo ufw status verbose
Estado: activo
Perfil predeterminado: deny (incoming), allow (outgoing)
Hasta      Acción      Desde
22/tcp     ALLOW IN    Anywhere
80/tcp     ALLOW IN    Anywhere
443/tcp    ALLOW IN    Anywhere

--- desde la notebook de un compañero, en otra red ---
$ nmap -Pn tudominio.me
PORT      STATE  SERVICE
22/tcp    open   ssh
80/tcp    open   http
443/tcp   open   https
3000/tcp  open   ppp
```

(3 puntos) Las dos salidas se contradicen. ¿Cuál de las dos describe el estado real del servidor y por qué? ¿Cuál es la conclusión operativa?

La que vale es nmap. Es la única que atraviesa el mismo camino que atravesaría un atacante. Cualquier herramienta que corra *dentro* del servidor está describiendo lo que ella cree.

Las dos salidas son correctas y no se contradicen realmente: ufw **sí** tiene esas tres reglas y ninguna abre el 3000; lo que pasa es que el 3000 lo publicó Docker y su tráfico no pasa por INPUT. ufw status no dice «el 3000 está cerrado»: dice «yo no tengo ninguna regla que lo abra». Son dos afirmaciones distintas y solo la segunda es la que ufw puede sostener.

Conclusión operativa: el panel sigue expuesto por el 3000. Hay que publicarlo bajo el dominio propio con HTTPS y recién entonces cerrar el acceso directo. Y la verificación de cierre del capítulo es la comprobación 5 (nmap desde afuera), no la 4 (ufw status): las demás describen *intenciones*, esa describe *hechos*.

Apunte: §2.8, §2.8.2 y §2.13

E3.

```
root@vps:~# uptime
21:14:03 up 3 hours, 12 min, 1 user

root@vps:~# grep -c "Failed password" /var/log/auth.log
1847
```

(3 puntos) El servidor se creó hoy a la tarde y su dirección no se le dio a nadie. Calculá la tasa aproximada por hora, explicá qué demuestra el número y decí qué medida del capítulo vuelve ese número irrelevante.

Tasa: $1847 \div 3,2 \text{ h} =$ unos **577 intentos fallidos por hora**, casi 10 por minuto, contra un servidor que existe hace tres horas y cuya dirección nadie publicó.

Qué demuestra: que el descubrimiento es **automático, permanente y no selectivo**. Nadie está atacando a este grupo: están atacando a todos, todo el tiempo, sin saber ni importarles qué hay del otro lado. Barrer el espacio IPv4 entero es una tarea programada, no una hazaña — y los resultados se indexan y se venden.

Qué lo vuelve irrelevante: PasswordAuthentication no. Sin contraseña habilitada no hay nada que adivinar por fuerza bruta, por muchos intentos que se acumulen. fail2ban ayuda —baja el ruido y el consumo— pero es una capa más, no la solución. Si el acceso fuera por contraseña, con esa tasa, alguno entraría.

Apunte: §2.9.1, §2.5.5 y §2.11.1

Cierre — una para pensar (sin puntaje)

El capítulo señala una forma de error que se repite en toda la ingeniería: **una herramienta te informó sobre sí misma y vos lo leíste como información sobre el sistema**. `ufw status` no dice «el 5432 está cerrado»; dice «yo tengo una regla que lo cierra».

Discutan en el grupo: ¿qué otras herramientas que usan todas las semanas les hablan de sí mismas y ustedes las leen como si hablaran del mundo? Piensen en el panel DNS de la Clase 1, en «los tests pasan», en «el build compiló», en «el deploy dice Success». En cada uno de esos casos, ¿cuál sería la verificación equivalente a correr `nmap` desde afuera?

No hay respuesta única. Lo que tiene que aparecer es la distinción entre **lo declarado** y **lo real**, y la idea de que la verificación válida es la que se hace *desde el mismo lugar que el usuario o el atacante*.

Paralelos que suelen salir y vale la pena rescatar: el panel de Namecheap muestra los registros que cargaste, no los que el mundo consulta (§1.7) — la prueba real es `dnschecker.org`. «Los tests pasan» dice que esos tests pasan, no que el sistema funciona — la prueba real es usar la aplicación. «El deploy dice Success» dice que el proceso terminó sin error, no que el servicio responde — la prueba real es abrir la URL desde otra red.

La regla general que conviene dejar dicha: **cada vez que una herramienta te reporte un estado, preguntate si te está hablando del mundo o de su propia configuración**.